

Bicycle Motion Tracking & State Estimation using Smartphone Sensor Fusion via Extended Kalman Filtering

Mid-Semester Project Evaluation Report – ECS323

Nikhil Vashisht (Roll No. 24228)

Vatsal Agarwal (Roll No. 24369)

September 27, 2026

Abstract

This report works through, from first principles, how we plan to estimate the motion of a bicycle using only the sensors already inside a smartphone: accelerometer, gyroscope, GPS, and magnetometer. Our course covers single-input single-output systems through transfer functions $G(s)$; this project needed something that could track several coupled physical quantities at once (position, speed, heading, and a slowly drifting sensor error) while handling the fact that bicycle motion is not linear. That pushed us toward state-space modeling and, because our kinematics involve $\cos \theta$ and $\sin \theta$, toward the *Extended* Kalman Filter (EKF) rather than the standard linear one. This report is written as a set of derivation notes rather than a textbook summary: every equation is built up from the physical picture of the problem, so that anyone with no prior Kalman filter background can follow how we got here and verify it themselves.

1 Problem Statement

We want to track a bicycle’s motion in 2D space using nothing but a phone’s onboard sensors. Each sensor gives us a different, incomplete, and noisy piece of the picture:

- **Accelerometer:** gives body-frame linear acceleration a_m . Fast (50–100 Hz) but if you integrate it twice to get position, small errors blow up quadratically within seconds.
- **Gyroscope:** gives angular velocity ω_m (yaw rate). Also fast, but it has an unknown, slowly-changing zero-rate offset — a **bias** b_ω — that we can’t read off the sensor directly, and integrating a biased rate drifts the heading estimate without bound.
- **GPS:** gives absolute position (P_x, P_y) and a ground speed v_{gps} . Accurate over long time scales, but slow (1–2 Hz), laggy, and can drop out entirely under tree cover or between buildings.
- **Magnetometer:** gives an absolute heading θ_{mag} from Earth’s magnetic field. Useful as an occasional heading anchor, but easily corrupted by nearby metal or electronics (hard-iron / soft-iron distortion).

None of these sensors alone gives a usable trajectory: the IMU alone drifts, GPS alone is too sparse and too slow for anything that wants to feel “real time.” The goal of the theory in this report is to combine them so that the fast IMU carries us between GPS fixes, and the slow-but-absolute GPS/magnetometer measurements periodically pull the estimate back toward the truth and, critically, let us also estimate the otherwise-invisible gyroscope bias b_ω .

2 What Do “State” and “State-Space” Actually Mean Here?

Our syllabus so far has treated a system as a black box described by a transfer function

$$G(s) = \frac{Y(s)}{U(s)}, \quad (1)$$

which tells you how an input $U(s)$ maps to an output $Y(s)$, but tells you nothing about what is happening *inside* the system to produce that mapping. That is fine for a single input driving a single output through a linear circuit or a linear mechanical system, but it breaks down for us for three concrete reasons:

1. We don’t have one input and one output. We have two inputs (a_m, ω_m) and up to four measurements $(P_{x,gps}, P_{y,gps}, v_{gps}, \theta_{mag})$ at once. That’s a multi-input multi-output (MIMO) problem, and $G(s)$ doesn’t naturally extend to that.
2. There is a quantity we care about — the gyroscope bias b_ω — that never appears directly in any sensor reading. It only reveals itself indirectly, through how the heading estimate behaves over time. A transfer function, which only relates input to output, has no place to put this.
3. Our kinematics involve $v \cos \theta$ and $v \sin \theta$. These are products of state variables, not something you can write as a ratio of polynomials in s . There is no Laplace transform for that.

Definition

The **state** of a system at time t , $\mathbf{x}(t)$, is the smallest set of numbers such that: *if you know $\mathbf{x}(t_0)$ and you know every future input $\mathbf{u}(\tau)$ for $\tau \geq t_0$, you can determine $\mathbf{x}(\tau)$ and the output $\mathbf{y}(\tau)$ for all $\tau \geq t_0$, without needing to know anything about what happened before t_0 .*

In plain words: the state is the system’s *memory*. It’s the minimum information you’d need to write down on a piece of paper so that someone else could pick up the simulation exactly where you left off.

The **state space** is simply the vector space $\mathbb{R}^n$ that $\mathbf{x}(t)$ lives in. A **state-space representation** describes a system as a first-order vector differential (or difference) equation

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) \quad (\text{or, nonlinearly, } \dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})) \quad (2)$$

$$\mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t) \quad (\text{or } \mathbf{y} = h(\mathbf{x})) \quad (3)$$

instead of the single scalar equation $Y(s) = G(s)U(s)$.

Why this matters: For us, the state is $\mathbf{x} = [P_x, P_y, v, \theta, b_\omega]^T$: position, speed, heading, and gyro bias. This is exactly the information we would need to write down to fully describe “where the bike is and how its sensors are currently misbehaving,” at any instant, without needing the entire sensor history before that instant. That’s the whole point of choosing a state-space model over a transfer function: it gives b_ω somewhere to live.

3 From Physical Motion to a Discrete-Time Model

3.1 Continuous-Time Kinematics

We model the bicycle (rider + bike together) as a point moving in the plane with a single forward speed $v(t)$ and heading $\theta(t)$, measured counter-clockwise from East. This is the standard “unicycle” kinematic simplification: it assumes the bike does not sideslip, i.e., its velocity vector always points along its heading. We are not modeling the bicycle’s wheelbase or steering angle

explicitly — the gyroscope’s measured yaw rate stands in directly for “how fast the heading is changing,” regardless of how that turning is mechanically produced.

Under this assumption, basic 2D velocity resolution gives:

$$\dot{P}_x(t) = v(t) \cos \theta(t) \quad (4)$$

$$\dot{P}_y(t) = v(t) \sin \theta(t) \quad (5)$$

The forward speed changes according to whatever forward acceleration the accelerometer is reporting (treated as a control input, not something with its own independent dynamics):

$$\dot{v}(t) = a_m(t) \quad (6)$$

The true heading rate is the gyroscope’s true angular velocity. The gyroscope doesn’t report this directly — it reports $\omega_m(t)$, which is the true rate plus a slowly-varying, unknown offset $b_\omega(t)$:

$$\omega_m(t) = \dot{\theta}(t) + b_\omega(t) \implies \dot{\theta}(t) = \omega_m(t) - b_\omega(t) \quad (7)$$

Finally, we don’t have a physical law for how b_ω evolves — it drifts slowly due to temperature and manufacturing imperfections, in a way we can’t predict in advance. The standard way to represent “we don’t know how this changes, but we know it changes slowly” inside a state-space model is a **random walk**:

$$\dot{b}_\omega(t) = w_b(t), \quad w_b(t) \text{ a small zero-mean noise process} \quad (8)$$

This is a modeling trick, not a physical law: it tells the filter “assume this is constant, but stay open to slowly revising that belief.”

Collecting these five equations is exactly the nonlinear continuous-time state-space model $\dot{\mathbf{x}}(t) = f_c(\mathbf{x}(t), \mathbf{u}(t))$ for our system, with $\mathbf{u}(t) = [a_m(t), \omega_m(t)]^T$.

3.2 Discretizing: From $\dot{\mathbf{x}} = f_c(\mathbf{x}, \mathbf{u})$ to $\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k)$

Our phone doesn’t give us continuous signals; it gives us samples every Δt seconds (with $\Delta t \approx 0.01\text{--}0.02\text{ s}$ for a 50–100 Hz IMU). The simplest way to turn a continuous ODE into a discrete update rule is **forward Euler integration**: approximate the derivative $\dot{\mathbf{x}}(t) \approx \frac{\mathbf{x}_k - \mathbf{x}_{k-1}}{\Delta t}$, which rearranges to

$$\mathbf{x}_k \approx \mathbf{x}_{k-1} + \Delta t \cdot f_c(\mathbf{x}_{k-1}, \mathbf{u}_k). \quad (9)$$

This is a first-order approximation: the error it introduces per step is $O(\Delta t^2)$, which is why it’s only reasonable when Δt is small — exactly our situation, since IMU sample intervals are on the order of 10–20 milliseconds. Applying this to each of the five continuous equations above gives us the discrete update equations we will actually use in the filter:

$$P_{x,k} = P_{x,k-1} + v_{k-1} \cos(\theta_{k-1}) \Delta t \quad (10)$$

$$P_{y,k} = P_{y,k-1} + v_{k-1} \sin(\theta_{k-1}) \Delta t \quad (11)$$

$$v_k = v_{k-1} + a_{m,k} \Delta t \quad (12)$$

$$\theta_k = \theta_{k-1} + (\omega_{m,k} - b_{\omega,k-1}) \Delta t \quad (13)$$

$$b_{\omega,k} = b_{\omega,k-1} + w_{b,k} \quad (14)$$

Why this matters: These are the same five equations as our first draft, but now they are not just “given” — they fall directly out of resolving velocity into x/y components and Euler-discretizing. The nonlinear terms $v_{k-1} \cos \theta_{k-1}$ and $v_{k-1} \sin \theta_{k-1}$ are exactly where a linear Kalman filter stops being valid, and exactly why we need the EKF’s linearization step in Section 5.

4 Our State Vector and Inputs — What We Track, and Why

$$\mathbf{x}_k = \begin{bmatrix} P_{x,k} \\ P_{y,k} \\ v_k \\ \theta_k \\ b_{\omega,k} \end{bmatrix}, \quad \mathbf{u}_k = \begin{bmatrix} a_{m,k} \\ \omega_{m,k} \end{bmatrix} \quad (15)$$

Symbol	Meaning	Units
$P_{x,k}$	Global East position	m
$P_{y,k}$	Global North position	m
v_k	Forward (body-frame) speed	m/s
θ_k	Heading / yaw angle	rad
$b_{\omega,k}$	Gyroscope zero-rate bias	rad/s
$a_{m,k}$	Measured forward acceleration (input)	m/s ²
$\omega_{m,k}$	Measured yaw rate (input)	rad/s

Why exactly these five states, and not more or fewer

- P_x, P_y are the whole point of the project — this is the trajectory we ultimately want to display.
- v, θ are needed because they are what actually drives P_x, P_y forward each step (Section 3.1); we can't propagate position without them.
- b_ω is included because, without it, an uncorrected gyro bias would silently corrupt θ forever. Including it as a state — rather than treating it as fixed — is what lets the filter learn and cancel it out using GPS/magnetometer corrections.

What we deliberately left out, for now

A more complete design would likely also track an **accelerometer bias** b_a (accelerometers drift too, just more slowly and with smaller effect on this particular model) and a **magnetometer hard-/soft-iron offset**. We chose not to include either yet:

- Accelerometer bias mostly affects v , and v is already being corrected every time a GPS fix arrives, so an uncorrected b_a is far less damaging here than an uncorrected b_ω , which corrupts θ (and therefore the direction the whole trajectory drifts in).
- We plan to handle magnetometer distortion with an offline/static calibration step (e.g., figure-8 calibration before riding) rather than estimating it online, at least for this stage.

This is explicitly a simplification for the mid-semester stage, not a final design decision — both are natural extensions once we have real logged data to see whether they matter in practice.

5 Why the Model Is Nonlinear, and How the EKF Linearizes It

5.1 The problem

Look again at $P_{x,k} = P_{x,k-1} + v_{k-1} \cos(\theta_{k-1})\Delta t$. This is a *product* of two state variables, v_{k-1} and $\cos(\theta_{k-1})$. A standard (linear) Kalman filter requires the update to be expressible as $\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k$ for some *constant* matrix $\mathbf{F}$ — but there is no constant matrix that reproduces $v \cos \theta$, because the relationship between input and output changes depending on where in state space you currently are.

5.2 The EKF's fix: linearize around the current estimate

The Extended Kalman Filter's core idea is: even though $f(\mathbf{x}, \mathbf{u})$ is nonlinear globally, it's approximately linear in a small neighborhood around our current best estimate $\hat{\mathbf{x}}_{k-1|k-1}$. We use a first-order Taylor expansion of f around that point, and use *that local linear approximation* in place of the true nonlinear function, just for one prediction step. The matrix that this Taylor expansion produces is the Jacobian:

$$\mathbf{F}_k = \left. \frac{\partial f(\mathbf{x}, \mathbf{u})}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_k}, \quad (\mathbf{F}_k)_{ij} = \frac{\partial f_i}{\partial x_j} \quad (16)$$

i.e., row i , column j of $\mathbf{F}_k$ tells you how much the i -th predicted state changes if you nudge the j -th *previous* state, holding everything else fixed. We compute every entry directly from the discrete equations of Section 3.2 (writing $\hat{v}, \hat{\theta}$ for the values at $\hat{\mathbf{x}}_{k-1|k-1}$):

$$\begin{aligned} \frac{\partial P_{x,k}}{\partial P_{x,k-1}} &= 1 & \frac{\partial P_{x,k}}{\partial P_{y,k-1}} &= 0 \\ \frac{\partial P_{x,k}}{\partial \theta_{k-1}} &= -\hat{v} \sin \hat{\theta} \Delta t & \frac{\partial P_{x,k}}{\partial b_{\omega,k-1}} &= 0 \\ \frac{\partial P_{y,k}}{\partial v_{k-1}} &= \sin \hat{\theta} \Delta t & \frac{\partial P_{y,k}}{\partial \theta_{k-1}} &= \hat{v} \cos \hat{\theta} \Delta t \\ \frac{\partial v_k}{\partial v_{k-1}} &= 1 & & \text{(all other partials of } v_k \text{ are 0, since } v_k \text{ only depends on } v_{k-1} \text{ and the input)} \\ \frac{\partial \theta_k}{\partial \theta_{k-1}} &= 1 & \frac{\partial \theta_k}{\partial b_{\omega,k-1}} &= -\Delta t \\ \frac{\partial b_{\omega,k}}{\partial b_{\omega,k-1}} &= 1 & & \end{aligned}$$

Assembling these into a matrix (rows = $P_x, P_y, v, \theta, b_\omega$; columns = same order):

$$\mathbf{F}_k = \begin{bmatrix} 1 & 0 & \cos \hat{\theta} \Delta t & -\hat{v} \sin \hat{\theta} \Delta t & 0 \\ 0 & 1 & \sin \hat{\theta} \Delta t & \hat{v} \cos \hat{\theta} \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & -\Delta t \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (17)$$

Note that $\mathbf{F}_k$ genuinely changes from step to step, because it depends on $\hat{\theta}$ and $\hat{v}$ — this is exactly what “linearizing around the *current* estimate” means, and it's the reason we need to recompute $\mathbf{F}_k$ fresh at every timestep rather than using one fixed matrix for the whole trajectory.

5.3 Measurement Jacobian $\mathbf{H}_k$

When GPS and magnetometer readings arrive together, the measurement is $\mathbf{z}_k = [P_{x,gps}, P_{y,gps}, v_{gps}, \theta_{mag}]^T$. Each of these reads a state variable *directly* (we are assuming, for now, $v_{gps} \approx v$, i.e., no sideslip — consistent with our unicycle assumption in Section 3.1), so $h(\mathbf{x}_k) = \mathbf{H}_k \mathbf{x}_k$ is already linear and its Jacobian is just:

$$\mathbf{H}_k = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \quad (18)$$

Notice $\mathbf{H}_k$ has *no column* for b_ω — no sensor measures it directly. This is precisely why we need the filter’s cross-correlations (built up through $\mathbf{F}_k$ over successive predict steps) to let GPS/magnetometer corrections indirectly teach the filter what b_ω must be. We verify this actually works in Section 8.

6 The EKF Recursion,

The filter alternates between two steps. We name every matrix here in plain language before using it, since these names get reused constantly.

6.1 Predict Step (runs every IMU sample, $\sim 50\text{--}100$ Hz)

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_k) \quad \text{push the last estimate forward using Section 3.2’s equations} \quad (19)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{Q}_k \quad \text{grow our uncertainty accordingly} \quad (20)$$

Here $\mathbf{P}_{k|k-1}$ is the **estimate error covariance** — a 5×5 matrix that represents *how confident we are* in each state and how our uncertainties in different states are correlated with each other. $\mathbf{Q}_k$ is the **process noise covariance**: it represents how much we trust our own motion model, i.e., how much unmodeled “surprise” (road bumps, imperfect Euler discretization, imperfect random-walk bias model) we expect to accumulate each step. A larger $\mathbf{Q}_k$ means “trust the model less, be more willing to be corrected by measurements.”

6.2 Update Step (runs whenever GPS/magnetometer data arrives, $\sim 1\text{--}2$ Hz)

$$\mathbf{y}_k = \mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1}) \quad \text{innovation: measured value minus what we expected to measure} \quad (21)$$

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k \quad \text{innovation covariance: how much spread we expect in that mismatch} \quad (22)$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T \mathbf{S}_k^{-1} \quad \text{Kalman gain: how much to trust the new measurement vs. our prediction} \quad (23)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k \quad \text{correct the state estimate} \quad (24)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} \quad \text{correct our confidence accordingly} \quad (25)$$

$\mathbf{R}_k$ is the **measurement noise covariance**, representing how much we trust the GPS/magnetometer readings themselves (larger $\mathbf{R}_k$ = noisier sensor = trust it less).

Why this matters: The Kalman gain $\mathbf{K}_k$ is really the entire idea of the filter in one matrix: it is computed automatically, every step, to be the mathematically optimal blend between “trust the model” and “trust the sensor,” based on how uncertain we currently are about each. We never hand-tune this weighting ourselves — we only choose $\mathbf{Q}$ and $\mathbf{R}$, and $\mathbf{K}_k$ falls out of the algebra.

7 Choosing Q and R (For This Stage)

Properly, $\mathbf{Q}_k$ should be derived from the IMU’s actual noise-density specifications (accelerometer and gyroscope datasheets typically report noise density in $\mu g/\sqrt{\text{Hz}}$ and $^\circ/\text{s}/\sqrt{\text{Hz}}$), propagated through the discretization. We are deferring that to the implementation phase, once we have a real device to characterize. For now, we use the simplest reasonable choice: a diagonal matrix,

$$\mathbf{Q}_k = \text{diag}(q_{P_x}, q_{P_y}, q_v, q_\theta, q_{b_\omega}), \quad (26)$$

diagonal because we have no principled reason yet to claim the process noise in, say, P_x and θ are correlated with each other — treating them as independent is the conservative default until data tells us otherwise. Similarly,

$$\mathbf{R}_k = \text{diag}(\sigma_{gps,x}^2, \sigma_{gps,y}^2, \sigma_{gps,v}^2, \sigma_{mag}^2) \quad (27)$$

with entries we will initially set from typical smartphone GPS/magnetometer accuracy figures (e.g., a few meters for consumer GPS) and then tune experimentally once we start logging real rides.

8 Is Our State Actually Observable? A Worked Computation

This section replaces our earlier draft, which *asserted* an observability conclusion without deriving it. We redo it properly here, for two concrete measurement scenarios, using the standard linear-systems observability matrix

$$\mathcal{O} = \begin{bmatrix} \mathbf{C} \\ \mathbf{CF} \\ \mathbf{CF}^2 \\ \vdots \\ \mathbf{CF}^{n-1} \end{bmatrix}, \quad n = 5, \quad (28)$$

applied *locally*, treating $\mathbf{F}_k \approx \mathbf{F}$ as momentarily constant (a standard and reasonable approximation for this kind of check, since $\mathbf{F}$ changes slowly relative to Δt). A state is observable if $\text{rank}(\mathcal{O}) = 5$.

8.1 Case 1: Heading-only updates (e.g., magnetometer available, GPS dropped out)

Here $\mathbf{C} = [0 \ 0 \ 0 \ 1 \ 0]$ (reads only θ). Multiplying row-vectors by $\mathbf{F}$ repeatedly:

$$\mathbf{C} = [0 \ 0 \ 0 \ 1 \ 0] \quad (29)$$

$$\mathbf{CF} = [0 \ 0 \ 0 \ 1 \ -\Delta t] \quad (\text{row 4 of } \mathbf{F}) \quad (30)$$

$$\mathbf{CF}^2 = [0 \ 0 \ 0 \ 1 \ -2\Delta t] \quad (31)$$

and in general $\mathbf{CF}^n = [0 \ 0 \ 0 \ 1 \ -n\Delta t]$. Every row of $\mathcal{O}$ has zeros in the P_x, P_y, v columns — those three states **never appear**, at any power, so they can never be recovered from heading measurements alone; this isn't something that “grows worse over time,” it's a hard structural fact from the first row onward. In the remaining (θ, b_ω) columns, rows $\mathbf{C}$ and $\mathbf{CF}$ are already linearly independent (as long as $\Delta t \neq 0$), so

$$\text{rank}(\mathcal{O}_{\text{heading-only}}) = 2. \quad (32)$$

Conclusion: with heading measurements alone, θ and b_ω are jointly observable — somewhat surprisingly, we can recover the gyro bias from heading alone, since the two rows above are independent equations in (θ, b_ω) . But P_x, P_y, v are structurally unobservable with no GPS: their *covariance* $\mathbf{P}_{k|k}$ (Section 6) will grow without bound over time, since nothing is correcting them — that is a statement about the Riccati recursion, a separate argument from the rank computation above, and our earlier draft had incorrectly blurred the two together.

8.2 Case 2: GPS + heading available

Here $\mathbf{H}$ (Section 5.3) already has rank 4, spanning P_x, P_y, v, θ directly, since its rows are literally the first four standard basis vectors. We only need to check whether one more row recovers b_ω . Take row 4 of $\mathbf{HF}$, i.e., row 4 of $\mathbf{F}$ itself: $[0 \ 0 \ 0 \ 1 \ -\Delta t]$. Subtracting $\mathbf{H}$'s own θ -row, $[0 \ 0 \ 0 \ 1 \ 0]$, from it gives

$$[0 \ 0 \ 0 \ 1 \ -\Delta t] - [0 \ 0 \ 0 \ 1 \ 0] = -\Delta t [0 \ 0 \ 0 \ 0 \ 1], \quad (33)$$

which is a nonzero multiple of the b_ω direction. So stacking just $\mathbf{H}$ and $\mathbf{HF}$ (two block rows, not all five) already spans all five coordinate directions:

$$\text{rank}\left(\begin{bmatrix} \mathbf{H} \\ \mathbf{HF} \end{bmatrix}\right) = 5. \quad (34)$$

Conclusion: with GPS and heading together, the full state — including the otherwise invisible b_ω — is locally observable in just one prediction step beyond the direct measurement. This is a much stronger and cleaner result than our original draft's unsupported claim, and it explains concretely *why* GPS fixes are what keep the filter's bias estimate from wandering off.

9 Planned Mobile Implementation

For the second half of the semester we intend to implement this filter as an Android app, most likely in native android using kotlin, streaming IMU data through native APIs and running the EKF prediction/update loop derived above. We have deliberately **not** finalized details of that architecture yet (exact threading model, matrix library choice, GPS/sensor polling strategy) — that planning is scoped for the next phase, once the estimator itself is validated against logged data.

10 Summary of Mid-Semester Progress

1. Derived the bicycle's continuous-time kinematic equations from first principles and obtained the discrete motion model via forward-Euler discretization, rather than presenting it as a given.
2. Defined and justified a 5-state model $[P_x, P_y, v, \theta, b_\omega]^T$, including an explicit account of which sensor error terms (accelerometer bias, magnetometer offset) we are *not* modeling yet, and why.
3. Derived every entry of the process Jacobian $\mathbf{F}_k$ term-by-term from the discrete equations, and the measurement Jacobian $\mathbf{H}_k$.
4. Worked through the full EKF predict/update recursion with each matrix's role explained in plain language.
5. Replaced our earlier, unsupported observability claims with an actual rank computation for two measurement scenarios, correctly separating the structural (rank-based) unobservability of P_x, P_y, v without GPS from the separate, covariance-growth argument for why that matters in practice.

11 Conclusion

This report walks, in order, from the physical motion of a bicycle to a fully derived nonlinear state-space model, through Jacobian linearization, to a complete Extended Kalman Filter recursion, and finally to a concrete check of which parts of that state we can and cannot actually recover from our sensors. The next phase of the project moves from this derivation to implementation: logging real IMU/GPS/magnetometer data from an actual ride, tuning $\mathbf{Q}$ and $\mathbf{R}$ against that data, and building the mobile application described briefly in Section 9.